Master of Information Technology — Independent Study (MIT4301)

Student Guideline

Overview

The Independent Study module gives you — and many of you join this conversion degree from non-IT academic backgrounds — the opportunity to turn technology you have already studied in theory into a small, working piece of software, built end-to-end, on your own initiative. By this stage of the programme you have covered the underlying computing concepts in your taught modules; what this module adds is the practical experience of applying that knowledge: setting up a real development environment, writing and testing running code, integrating tools and services, and shipping something that genuinely works.

The scope of the software itself is intentionally narrow — a small, well-bounded project rather than a large system. What matters is the journey and its output together: learning skills independently, applying them correctly, and being able to defend the resulting code as genuinely your own. Instead of an individually assigned supervisor, you'll have a weekly, cohort-wide technical session that walks you through the practical skill each stage of the project needs, and a single, rigorously assessed Final Viva where your knowledge, your working software, and your code are all examined directly. Because the code and its authorship are examined this closely, treat this as more demanding at the point of assessment than a lightly-supervised project, not less.

This guide explains what each task requires and which learning outcomes it develops, so you understand why you are doing each task, not only what is expected of you.

Learning Outcomes (ILOs)

On completing this module, you will be able to:

- ILO1. Gain in-depth knowledge of concepts and technologies in IT.
- ILO2. Develop specific skills by applying the technologies to a particular situation.
- ILO3. Gain effective communication skills.
- ILO4. Develop initiative, self-directed learning, and effective time and project management skills.
- ILO5. Enhance critical thinking and problem-solving skills.

Topic Selection and Weekly Technical Sessions

The Module Coordinator publishes a list of topics ahead of Week 1. You select one topic on a first-come, first-served basis; up to five students may select the same topic, provided each of you pursues a distinct approach, confirmed with the Module Coordinator when your proposal is reviewed.

Instead of an individually assigned supervisor, the module is delivered through a weekly, cohort-wide Technical Session, held in a dedicated weekend timeslot. Each session is led by a Facilitator with relevant expertise — a different Facilitator may lead different sessions across the module — and takes the form of a short technical demonstration or guided walkthrough of a specific implementation skill: for example, connecting an application to a SQL or NoSQL database, or turning a wireframe into a working user interface. You've already covered the underlying theory for these topics elsewhere in the programme; the session's purpose is the practical "how," not a re-teaching of the concept.

Each session opens with a brief round in which everyone reports their status in one line — on track, or stuck on a specific problem — followed by a small number of students, chosen at random, briefly showing evidence of their current progress: a running piece of code, recent commits, or the in-progress chapter. Use this as a genuine early-warning opportunity: raise a problem here rather than letting it surface for the first time at the Viva.

Task–Outcome Map

The table below makes explicit which ILO(s) each task is designed to develop, so you understand why you are doing each task.

Task	Nature of Task	Primary ILO(s)	Marks
Proposal	Planning / approval gate	ILO4	10
Timely Submission of Weekly Chapters (10)	Progressive documentation of the learning journey, submitted as it happens	ILO1, ILO2, ILO3, ILO4, ILO5	20
Final Viva — Knowledge Presentation	Communication of understanding of IT concepts and technologies	ILO1, ILO3	25
Final Viva — Application Demo & Code Review	Applied technical skill, verified live; authorship and understanding of your code	ILO2, ILO5	25
Final Viva — Portfolio & Chapters Defense	Critical reflection on the learning journey, examined collectively	ILO3, ILO5	20

Assessment Structure

Assessment is organised around three components:

The Proposal: is an approval gate — you cannot proceed to the chapter-writing stage without approval from the Module Coordinator, who also confirms your approach is sufficiently distinct if others share your topic.

Minimum criteria for approval:

- At least 2 distinct use cases — the system must support two or more meaningful user actions/functions (e.g. "register a loan" and "process a return," not just one CRUD screen repeated).
- A persistent data store — the project must use a real database (SQL or NoSQL, local). In-memory-only data or hardcoded values do not satisfy this.
- PHP and MySQL are the recommended technology stack for web-based projects. Alternative technologies may be used if they are appropriately justified and feasible within the 10-week development period.
- A clearly bounded scope — what's in and out of scope must be explicit, and sized for one student to build and defend alone within the module's timeframe (not a multi-role, enterprise-scale system).

Timely Submission of Weekly Chapters: rewards submitting each of your ten weekly chapters on time as a genuine, evidence-based account of that stage of the journey, rather than reconstructed later. Each chapter carries 2 marks for timely, genuine submission; the full evaluation of your chapters' content happens together with the rest of your work at the Final Viva.

The Final Viva: is a single, consolidated evaluation event, carrying 70% of the total mark. This is where you bring together your knowledge, your working software, and your portfolio, and defend all three live in front of a panel. The Viva runs across Weeks 14 and 15 to accommodate the full cohort.

Within the Final Viva, the Application Demo & Code Review component includes a Code Review element that acts as a gate as well as a mark.

Code Review

The Code Review element of your Application Demo is scored out of 10 marks, against three things the panel will check:

Explaining your code (4 marks): you can walk the panel through any part of your submitted code, line by line, on request.

Live modification and debugging (4 marks): you can make a small change to your code, or diagnose and fix a small issue the panel introduces, in real time.

Code quality (2 marks): your code reflects the practices covered in the weekly Technical Sessions (readability, structure, and sound use of version control).

The Weekly Portfolio

Weeks 1–2 are used for topic selection, feasibility study, background study, and proposal submission; there is no portfolio chapter for these weeks. You then build a portfolio of your software journey one chapter at a time, from Week 3 to Week 12. Each chapter is a short, honest account of that stage of the journey — the value is in showing genuine progression: what was unfamiliar, what was learned, what went wrong, and what was figured out.

Suggested themes for each chapter are given below.

Wk	Chapter Theme	What You Write / Show
3	Why this topic? Background study and designing a solution	Your starting point — what drew you to the topic, and what's unfamiliar going in. Analysing the requirements and designing a solution.
4	Learning the concepts	Your understanding of the Information Technology concepts involved.
5	Getting my tools working	The (often messy) experience of installing/setting up software and a repository — what confused you, what you had to look up.
6	First small step	Evidence of the first bit of working code or output, however small, and how you got there.
7	Hitting a wall	A real problem you got stuck on, and how you solved it (or are still solving it).
8	Learning something new mid-way	A new tool, technique, or concept picked up that you didn't know in Week 3.
9	Making it usable	How you made the thing usable/interactable, described simply.
10	What broke and what I fixed	Bugs, failures, or redesigns — normalising iteration as part of learning.
11	Evidence of progress	A simple repository activity snapshot — "here's what I've built over time."
12	How I've changed since I started	Direct reflection on what you know now that you didn't, in your own words.

At the Viva, these ten chapters — together with your repository and working software — form the basis of your portfolio defense. You walk the panel through a journey: where you started, what you learned, what you built, and what you would do differently.

Timeline

Wk	Weekly Technical Session	Deliverable Due This Week
1	Writing the project proposal	Topic selection and feasibility study underway
2	Background study; designing a solution; ethics of AI	Proposal submitted — approval gate (Module Coordinator)
3	Wireframe to UI; UX heuristics	Chapter 1 — Why this topic? Background study and designing a solution
4	Form validation; environment setup	Chapter 2 — Learning the concepts
5	Repository setup; database setup	Chapter 3 — Getting my tools working
6	First feature implementation; Git workflow	Chapter 4 — First small step
7	Unit testing; systematic debugging	Chapter 5 — Hitting a wall
8	Integrating a new feature/tool/library/API; integration testing	Chapter 6 — Learning something new mid-way
9	System testing; regression testing	Chapter 7 — Making it usable
10	Refactoring; safe bug-fixing	Chapter 8 — What broke and what I fixed
11	Reading repository history as evidence; writing critical self-reflection	Chapter 9 — Evidence of progress
12	Portfolio assembly and structuring	Chapter 10 — How I've changed since I started
13	Viva preparation	Consolidation — assemble all ten chapters into your final portfolio
14–15	—	Final Viva (split across two weeks to accommodate the full cohort)

Portfolio Format

Length: approximately 30–40 pages in total, once front matter and supporting evidence are included. The ten weekly chapters themselves are kept short (inclusive of screenshots and inline images).

Front matter: Title page, table of contents, and a list of figures if a meaningful number of screenshots/diagrams are included.

Introduction chapter: Before the weekly chapters, write a short introduction to yourself — your background, why you chose this degree and this topic, and what you personally want to get out of it. State your own goals and objectives for the module in your own words, not a generic project introduction.

Body: The ten weekly chapters, included largely as submitted, with inline screenshots, code snippets, and other evidence woven into the relevant chapter rather than collected separately.

Closing chapter (Conclusion): An expanded version of your Week 12 reflection, drawing together what changed across the whole journey.

References: Harvard style.

Evidence base: Repository commit history should be embedded as figures or screenshots within the relevant chapters, rather than attached as separate stand-alone deliverables.

Use of AI Tools

AI tools (e.g. code assistants, chatbots) may be used during this module, but their use is governed by one principle: you must be able to explain, modify, and debug any code you submit, regardless of whether AI helped produce it. This module exists to build your own understanding, and outsourcing that understanding to a tool defeats its purpose. This principle, and how to apply it responsibly, is covered directly in your Week 2 Technical Session — so the following applies:

- AI tools may be used the way a textbook, documentation, or forum answer would be used — to explain unfamiliar syntax, suggest boilerplate, or help debug an error you have already tried to diagnose.
- AI tools may not be used to generate a solution you do not understand well enough to explain, modify, or debug unaided. If you cannot walk through what a block of code does, line by line, do not submit it.
- Each weekly portfolio chapter should include a brief, honest note on where you used AI that week and what you had to learn or verify independently to understand it. This keeps your AI use visible across the journey rather than something that surfaces only at the Viva.
- At the Viva, the Application Demo & Code Review may probe any line of code, AI-assisted or not. Being unable to explain AI-assisted code is treated the same as being unable to explain any other code you submit — including the risk of failing the Code Review gate described above.

Agree an explicit minimum viable version of your project with the Module Coordinator at the proposal stage — what the Week 14–15 Viva deliverable looks like at its simplest, working form. This protects you from scope creep and unpaid overtime chasing a more polished version than the module requires.

Proposal Template

Formatting: Times New Roman. Main headings 18pt, subheadings 16pt, other headings (optional) 14pt, normal text 12pt. Margins: top/right/bottom 2cm, left 3cm.

1. **Project Title** — A brief introduction to the project, properly describing it. You may also include your personal motivation for undertaking the project.
2. **Background and Problem Understanding** — Briefly explain the problem/domain addressed by the assigned project and demonstrate the student's understanding of the need for the proposed system.
3. **Project Objectives** — State the specific objectives to be achieved through the system development.
4. **Proposed System** — Describe the student's initial understanding of the system, including intended users, major functions, expected inputs and outputs, and overall operation of the system.
5. **Project Scope** — Clearly define what will and will not be implemented within the project.
6. **Proposed Technologies and Tools** — Identify the technologies the student intends to use and briefly justify their suitability.
7. **Proposed Development Approach** — Briefly describe how the student intends to carry out: requirements analysis → design → implementation → testing → evaluation.
8. **Expected Deliverables** — E.g. functional information system, source code, portfolio.
9. **Work Plan / Timeline** — A simple week-by-week or milestone-based schedule.
10. **References** — Relevant technical documentation, standards, articles, books, or similar systems consulted in preparing the proposal.

Length of the report: 2–4 pages. Since the coordinators assign the title, the proposal can mainly be used to assess the student's understanding of the assigned project, proposed technical solution, appropriateness of technologies, scope, and feasibility of the work.

Sample Topics

Web and Information Systems

1. **Appointment and Queue Management System** — Develop a system for appointment scheduling, queue allocation, notifications, and basic reporting for a clinic or service centre.
2. **Laboratory Equipment Reservation System** — Develop a system for reserving shared laboratory equipment, managing availability, recording usage, and reporting utilization.
3. **Vehicle/Transport Booking and Tracking System** — Manage shared vehicle reservations, driver assignment, trip logging, and maintenance schedules.
4. **Volunteer/Shift Scheduling System** — Manage volunteer sign-ups, shift slots, conflict checking, and attendance confirmation.
5. **Help Desk and IT Support Ticketing System** — Develop a ticketing system with issue categorization, assignment, priority, status tracking, comments, and dashboards.

6. **Product Return and Warranty Claim System** — Handle return requests, warranty validation, status tracking, and resolution history.

7. **Inventory and Asset Management System** — Track organizational assets, locations, users, maintenance history, transfers, and disposal.

8. **Library/Resource Lending System** — Track book/equipment loans, due dates, renewals, overdue notifications, and borrower history.

9. **Lost and Found Management System** — Log found/lost items with descriptions and photos, matching suggestions, and claim verification workflow.

10. **Event Registration and Attendance Management System** — Support event creation, participant registration, QR-code attendance, certificates, and summary reports.

11. **Visitor and Access Management System** — Register visitor check-in/check-out, host approval, badge/QR generation, and visit history logs.

12. **Community Notice Board System** — Post announcements/notices by category, with approval workflow, expiry dates, and read/acknowledgement tracking.

13. **Research Project Management System** — Manage student research projects, supervisors, milestones, document submissions, progress updates, and feedback.

14. **Internal Job Application Tracking System** — Track internal vacancy postings, applications, interview stages, and hiring decisions.

15. **Membership and Subscription Management System** — Manage member sign-ups, renewal reminders, payment status tracking, and membership tiers.

16. **Donation and Fundraising Management System** — Record donations, donor details, campaign progress, and generate receipts/reports.

17. **Simple Job/Task Board** — Post tasks or gigs, allow applications, track status (open/assigned/done), with basic filtering.

18. **Course Material and Assignment Repository** — Upload/organize course materials by subject and week, track submission deadlines, and log downloads/views.

19. **Simple E-Learning Quiz and Grading System** — Create quizzes, allow submissions, auto-grade objective questions, and show score history.

20. **Document Approval Workflow System** — Implement submission, review, approval/rejection, comments, notifications, audit trail, and role-based access.